

ACCELERAZIONE DI SIMULAZIONI MONTE CARLO APPLICATE AD UN ASSET FINANZIARIO

...

**Maurizio Amadori, matricola 0001243119
Andrea Simonetti, matricola 0001221864**

COS'È UNA SIMULAZIONE MONTECARLO?

Metodo statistico frequentemente usato in vari ambiti:

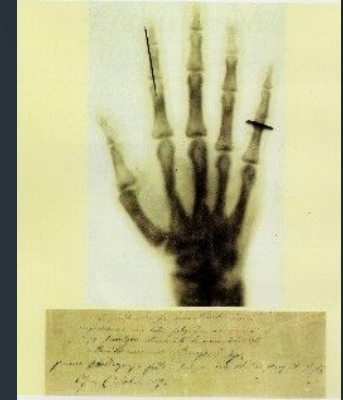
❖ *Finanza e Assicurazioni*

- *Valutazione di opzioni*
- *Gestione del portafoglio*
- *Pianificazione previdenziale*



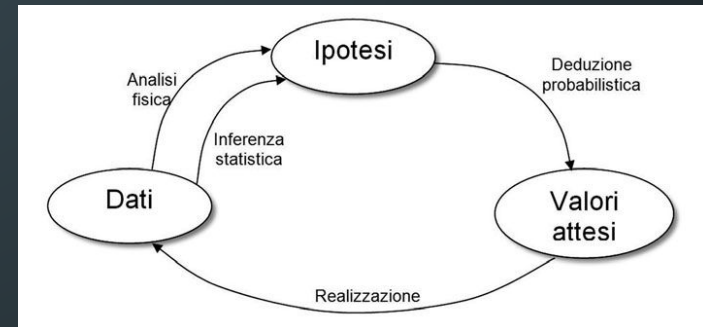
❖ *Industria*

- *Analisi della Tolleranza*
- *Affidabilità e Manutenzione*
- *Analisi dei Rischi*



❖ *Fisica e Scienza dei Materiali*

- *Fisica delle particelle*
- *Radioterapia*



IL PROBLEMA

Investendo una somma x in un asset finanziario in un istante T_0 , quale valore ci si può aspettare di ottenere dal mio investimento ad un istante T_1 ?

**Per provare a rispondere si può usare il metodo
Monte Carlo!**

DINAMICA STOCASTICA DEI PREZZI

IL MOTO BROWNIANO GEOMETRICO (GBM)

IPOTESI FONDAMENTALE

L'evoluzione del sottostante S_t segue un processo descritto dalla seguente Equazione Differenziale Stocastica (SDE):

LEGENDA

- ❑ **Z** Variabile casuale standard $N(0,1)$
- ❑ **S_t** Prezzo dell'indice al tempo t .
- ❑ **μ** Drift (tendenza media), del dataset.
- ❑ **σ** Volatilità storica annualizzata.
- ❑ **W_t** Processo di Wiener (Moto Browniano)

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

$$\text{Prezzo}_{t+\Delta t} = \text{Prezzo}_t \cdot e^{\left[\left(\mu - \frac{1}{2}\sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z \right]}$$

*Usando i log rendimenti
si possono sfruttare le loro proprietà*

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

$$\ln(P_{t+1}/P_0) = \sum_{i=0}^t r_{\log,i}$$

IL METODO MONTE CARLO

Si basa sulla **Legge dei Grandi Numeri** (LLN)

Il **valore atteso** di una funzione $f(S_t)$ viene *approssimato* dalla media campionaria di N realizzazioni indipendenti:

$$\mathbb{E}[f(S_T)] \approx \frac{1}{N} \sum_{i=1}^N f(S_{T,i})$$

L'**errore standard** dello stimatore Monte Carlo *decade* secondo la proporzione:

$$\epsilon \propto \frac{1}{\sqrt{N}}$$

MONTE CARLO: SIMULAZIONI SEQUENZIALI NAIF CPU

```
// Parametri Iniziali
double driftTerm = (drift - 0.5 * volatilita * volatilita) * T_YEARS;
double volTerm = volatilita * std::sqrt(T_YEARS);

for (int i = 0; i < nSimulations; ++i) {
    double Z = distribution(generator); // Generazione numero casuale
    double ST = S0 * std::exp(driftTerm + volTerm * Z); // Formula GBM
    simulatedPrices[i] = ST;
}
```

MONTE CARLO: SIMULAZIONI ACCELERATE NAIF GPU

```
__global__ void monteCarloKernel(double* out,int n,double S0,double
driftTerm,double volTerm,unsigned long seed) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    curandStatePhilox4_32_10_t state;
    curand_init(seed, idx, 0, &state);

    double Z = curand_normal_double(&state);
    out[idx] = S0 * exp(driftTerm + volTerm * Z);
}
```

MONTE CARLO: SIMULAZIONI ACCELERATE NAIF GPU

```
double driftTerm = (drift - 0.5 * volatilita * volatilita) * T_YEARS;
double volTerm    = volatilita * std::sqrt(T_YEARS);
// Allocazione variabile su GPU per simulazioni
double* d_sim;
cudaMalloc(&d_sim, N_SIMULATIONS * sizeof(double));
// Definizione griglia e blocchi 1D e 1D
dim3 blockDim(256, 1, 1);
dim3 gridDim((N_SIMULATIONS + blockDim.x - 1) / blockDim.x, 1, 1);

monteCarloKernel<<<gridDim, blockDim>>>(d_sim, N_SIMULATIONS, S0, driftTerm, volTerm, SEED);
cudaDeviceSynchronize();
// Trasferimento prezzi simulati da GPU a CPU
std::vector<double> simulatedPrices(N_SIMULATIONS);
cudaMemcpy(simulatedPrices.data(), d_sim, N_SIMULATIONS * sizeof(double), cudaMemcpyDeviceToHost);
cudaFree(d_sim);
```


SOLUZIONE GPU (ITERAZIONI 1, 3)

```
__global__ void monteCarloKernel(float* out, int n, float
S0, float driftTerm, float volTerm, unsigned long seed) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    curandStatePhilox4_32_10_t state;
    curand_init(seed, idx, 0, &state);
    float Z = curand_normal(&state);
    out[idx] = S0 * exp(driftTerm + volTerm * Z);
}
```

L'ordinamento ha tempi rilevanti al pari della simulazione Monte Carlo

- ❖ Sostituzione tipi double con float
- ❖ Aggiunto sort parallelo con libreria CUDA thrust

```
// Wrapping del puntatore raw per thrust
thrust::device_ptr<float> dPtr(dSim);
thrust::sort(dPtr, dPtr + nSimulations);
```

SOLUZIONE GPU (ITERAZIONI 4, 5)

Semplificazione operazioni e PARALLELIZZAZIONE

```
curandStatePhilox4_32_10_t state;
curand_init(seed, tid, 0, &state);

// Genero 4 numeri casuali in un colpo solo (istruzione vettoriale)
float4 Z = curand_normal4(&state);

float4 res;
// Funzione esponenziale approssimata
res.x = S0 * __expf(driftTerm + volTerm * Z.x);
res.y = S0 * __expf(driftTerm + volTerm * Z.y);
res.z = S0 * __expf(driftTerm + volTerm * Z.z);
res.w = S0 * __expf(driftTerm + volTerm * Z.w);

// Scrittura vettorizzata in memoria globale (1 transazione per 4 float)
reinterpret_cast<float4*>(out)[tid] = res;
```

SOLUZIONE GPU (ITERAZIONI 6, 8, 9)

Aumentate le simulazioni per thread

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
int totalThreads = gridDim.x * blockDim.x;
// Ogni thread ha il proprio stato curand
curandStatePhilox4_32_10_t state;
curand_init(seed, tid, 0, &state);
float4 Z;
float4 res;
for (int i = 0; i < N_CYCLES; i++) {
    int globalFloat4Idx = tid + (i * totalThreads);
    // Controllo bounds (moltiplicato per 4 perché n è il numero di float, non float4)
    if (globalFloat4Idx * 4 >= n) return;
    Z = curand_normal4(&state);
    res.x = S0 * __expf(driftTerm + volTerm * Z.x);
    res.y = S0 * __expf(driftTerm + volTerm * Z.y);
    res.z = S0 * __expf(driftTerm + volTerm * Z.z);
    res.w = S0 * __expf(driftTerm + volTerm * Z.w);
    reinterpret_cast<float4*>(out)[globalFloat4Idx] = res;}
```

SOLUZIONE GPU (ITERAZIONE 7)

VICOLI CIECHI TENTATI:

- ❖ Pinned Memory
- ❖ Kernel Separati per puro calcolo e allocazione risorse

A cosa serve la pinned memory se dovremmo ordinare i dati?

Lanciare due kernel sequenziali è costoso se non puoi fare latency hiding

JUST KEEP IT SIMPLE.

Come si possono alleggerire i trasferimenti di memoria?

SOLUZIONE GPU (ITERAZIONI 9, 10)

Alleggerito il trasferimento di dati *device to host*

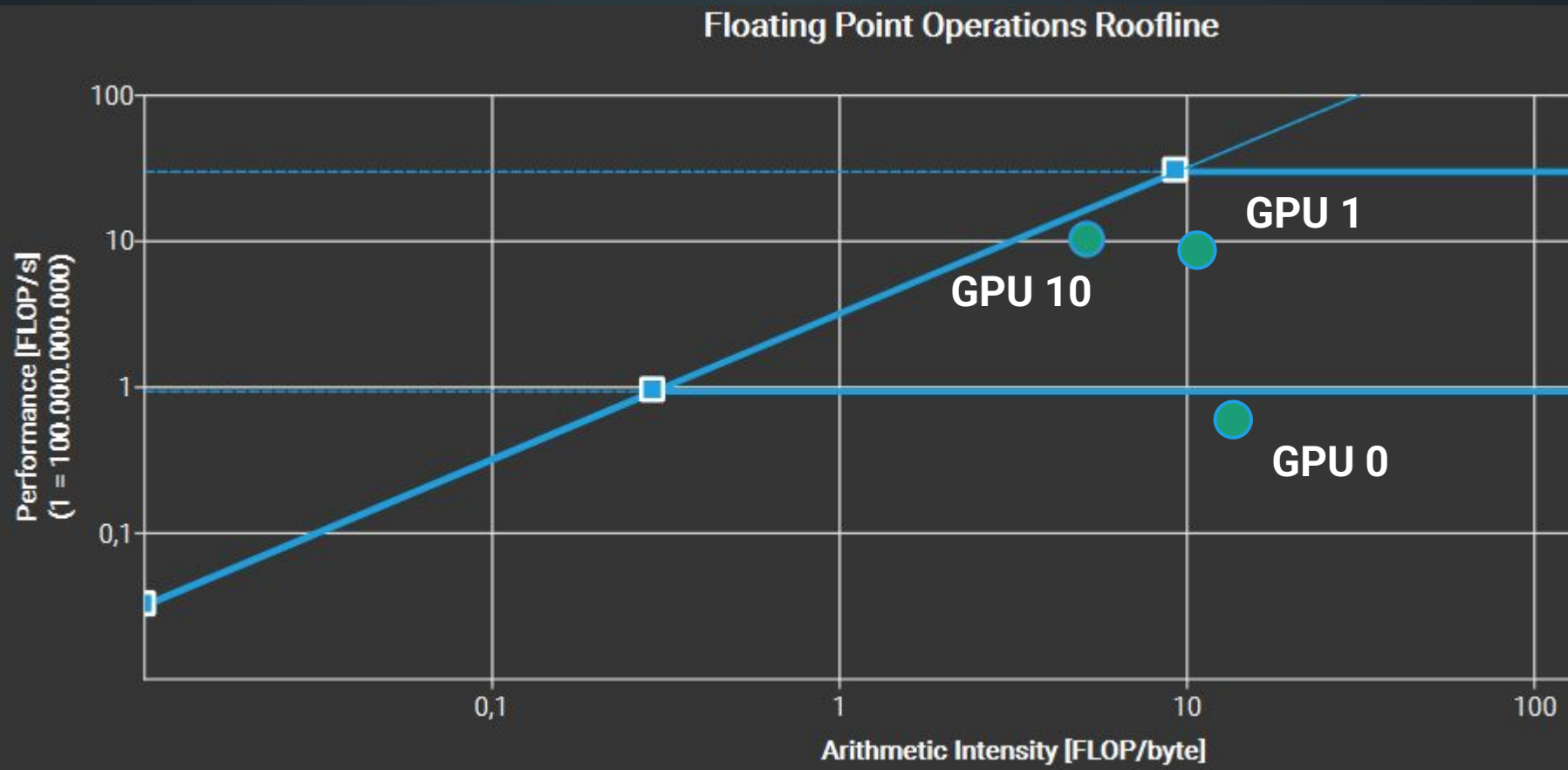
```
// dSimDevice = puntatore all'array sul device che abbiamo appena ordinato
// Analisi dei risultati
float priceWorst, priceMed, priceBest;

int idxWorst = (int)(nSimulations * 0.01f);
int idxMed    = (int)(nSimulations * 0.50f);
int idxBest   = (int)(nSimulations * 0.99f);

// Copia da (dSimDevice + offset) a variabile CPU
cudaMemcpy(&priceWorst, dSimDevice + idxWorst, sizeof(float), cudaMemcpyDeviceToHost);
cudaMemcpy(&priceMed,    dSimDevice + idxMed,    sizeof(float), cudaMemcpyDeviceToHost);
cudaMemcpy(&priceBest,   dSimDevice + idxBest,   sizeof(float), cudaMemcpyDeviceToHost);
```

Trasferire tutto l'array è molto più lento rispetto a solo i valori di interesse

ROOFLINE MODEL

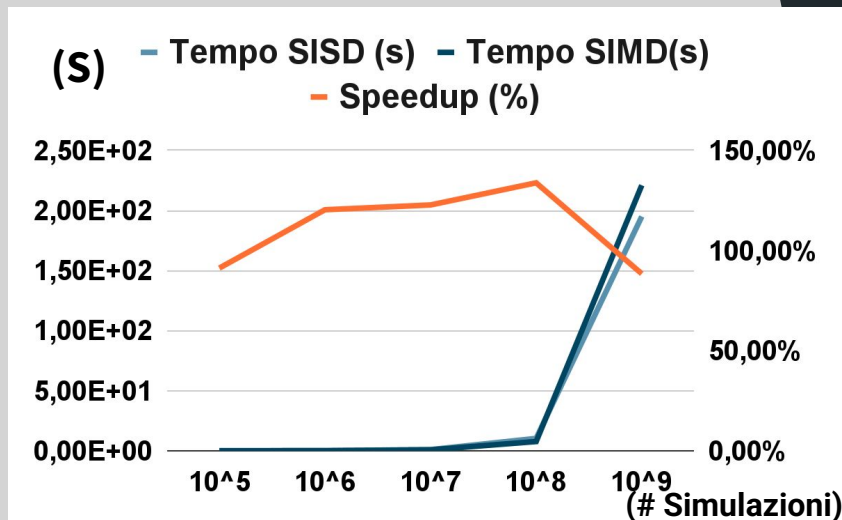


SOLUZIONE SIMD

Soluzione sequenziale naif come base e sostituzione
dell'algoritmo RNG C++ con Xorshift

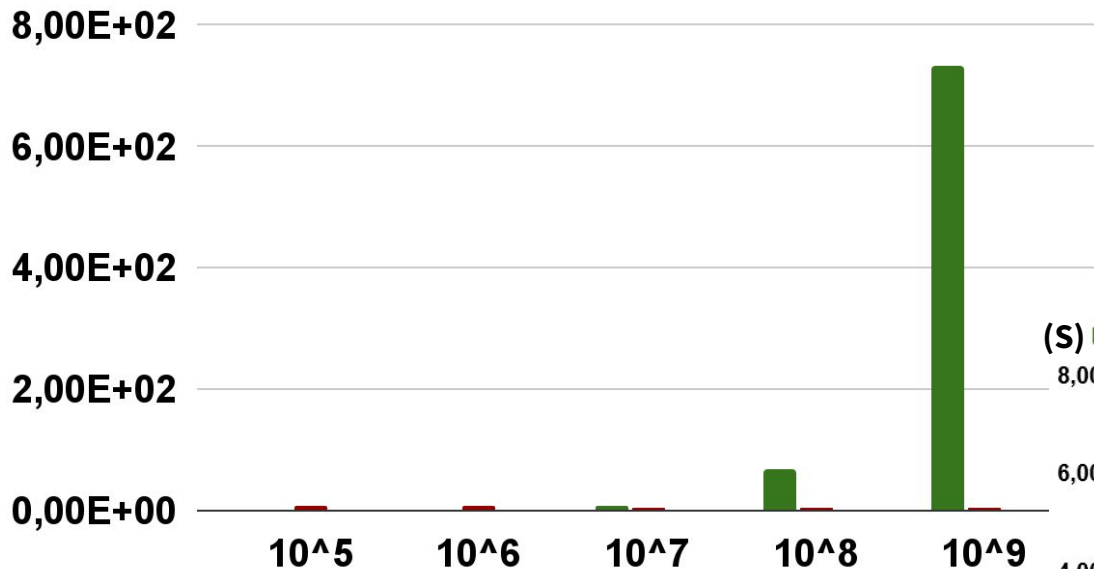
```
// Loop Principale
for(long i = 0; i + 3 < nSimulations; i += 4){
    __m128 Z = normalSIMD(rngState);
    __m128 X = _mm_add_ps(driftV, _mm_mul_ps(volV, Z));
    _mm_store_ps(out, X);
    for(int k = 0; k < 4; ++k){
        out[k] = S0 * std::exp(out[k]);
    }
    __m128 ST = _mm_load_ps(out);
    _mm_storeu_ps(&simulatedPortfolioValues[i], ST);
}

// Stato random per parte sequenziale
uint32_t seqState = 0xA2B3C4Du + nSimulations;
// Tail sequenziale finale (da 1 a 3 simulazioni)
for(long i = (nSimulations & ~3); i < nSimulations; ++i){
    float Z = normalSeq(seqState);
    simulatedPortfolioValues[i] = S0 * std::exp(driftTerm + volTerm * Z);}
```

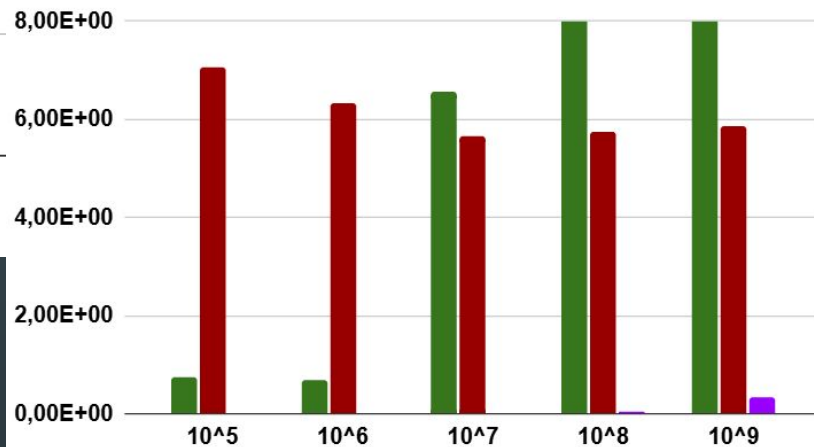


RISULTATI OTTENUTI E ANALISI

(S) ■ Tempo CPU ■ Tempo GPU Naive ■ Tempo GPU Opt



(S) ■ Tempo CPU (s) ■ Tempo GPU Naive (s) ■ Tempo GPU Opt (s)

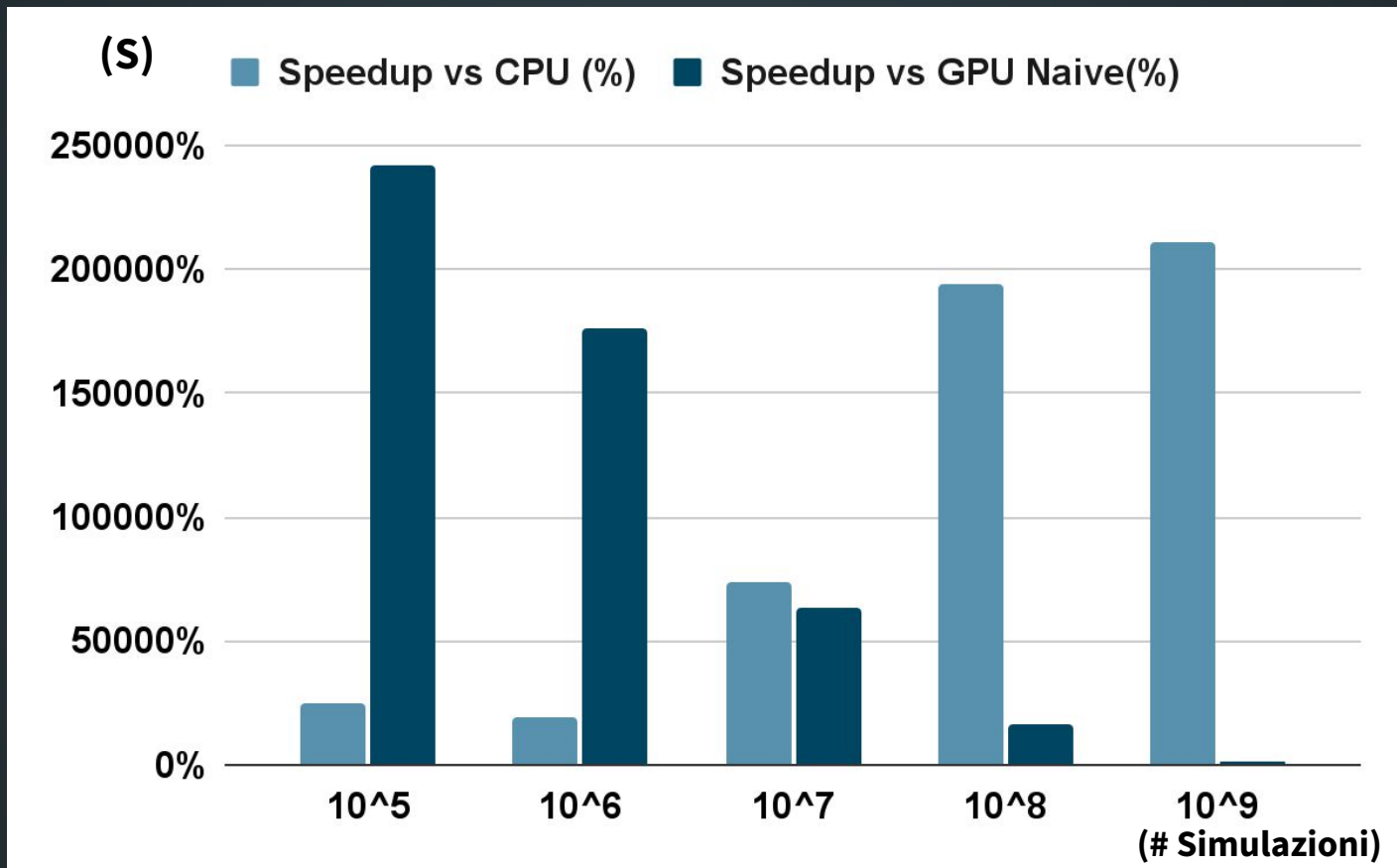


(# Simulazioni)

zoom

(# Simulazioni)

RISULTATI OTTENUTI E ANALISI: GPU OTTIMIZZATA



ALTRE STRATEGIE E ASPETTI DA CONSIDERARE

Per alcuni asset finanziari più particolari il calcolo del rendimento non è così banale. Questo può voler dire dover implementare simulazioni Monte Carlo Path Dependent

```
for (int i = 0; i < nSimulations; ++i) Naive CPU Path Dependent
{
    double currentPrice = S0;
    // Loop interno: cammina giorno per giorno
    for (int day = 0; day < DAYS_OPEN_IN_YEAR * T_YEARS; ++day)
    {
        double Z = distribution(generator);
        // Formula esponenziale incrementale
        currentPrice = currentPrice * std::exp(driftStep + volStep * Z);
    }
    // Salviamo il valore dell'indice raggiunto dalla simulazione
    simulatedPortfolioValues[i] = currentPrice;
}
```



Ogni nuovo valore
dipende dal precedente

GRAZIE MILLE PER L'ATTENZIONE



Maurizio Amadori, matricola 0001243119
Andrea Simonetti, matricola 0001221864

ESTENSIONE PROGETTO
ACCELERAZIONE DI SIMULAZIONI
MONTE CARLO
APPLICATE AD UN ASSET FINANZIARIO



Maurizio Amadori, matricola 0001243119
Andrea Simonetti, matricola 0001221864

COSA RIGUARDA L'ESTENSIONE DEL PROGETTO?

- 1) Implementare generatori alternativi a cuRAND
- 2) Ottimizzare i generatori per l'esecuzione su GPU
- 3) Confrontare le prestazioni con quelle di cuRAND (tempo di esecuzione e qualità statistica delle sequenze)

GENERATORI ALTERNATIVI ESPLORATI

PCG32

SplitMix64

Xoroshiro128+

Xorshift64*

ARCHITETTURA E FUNZIONAMENTO DI UN GENERATORE

- Definizione **stato** interno **generatore**
- Funzione di inizializzazione (**init**)
- Funzione di generazione di un nuovo numero (**step**)
- Generazione parallelizzata (GPU)

CuRAND

- Builtin del CUDA Framework
- Basato sulla famiglia di generatori Xorshift
- Ottima qualità di sequenze
- Xorwow sotto la scocca

XORSHIFT

- Registri a scorrimento a retroazione lineare (LFSR)
- I dati in ingresso sono prodotti da una trasformazione lineare su $GF(2)$ dello stato interno
- Applicazione ripetuta di XOR e SHIFT
- Efficienza, con uso contenuto di polinomi sparsi
- Deboli se applicati senza alcuna rifinitura
- Le versioni + e * applicano uno scramble, non-linearità
- Xoroshiro aggiunge la rotazione dei bit
- Xorwow usata in CUDA, usa uno scramble aggiungendo allo stato un contatore modulo 2^{32}
- Consigliato inizializzare lo stato con un generatore differente (non partire da uno stato all-zero)

PCG32

- Genera interi a 32 bit (nel test si concatenano due interi generati)
- Ha come base generatore lineare congruenziale
- **Combinazione lineare algebrica su modulo**

$$X_{n+1} = (aX_n + c) \% m$$

con periodo completo se:

* c e m sono coprimi

* $a - 1$ è divisibile per tutti i fattori di m

* $a - 1$ è un multiplo di 4 se m è un multiplo di 4

- Si aggiunge una trasformazione dell'output (nel nostro caso, Xorshift RR)
- **Qualitativamente pessimo se non inizializzato correttamente**

SPLITMIX64

- Default di Java
- Non indicato per scopi crittografici
- Molto veloce
- Combina somme, moltiplicazioni con XOR e SHIFT, avvalendosi di “magic numbers” che garantiscono qualità
- Adatto per inizializzare correttamente altri generatori (seeding)

COME ACCELERIAMO I GENERATORI IMPLEMENTATI?

- 1) Sostituiamo CuRand con la specifica implementazione
- 2) Adattiamo l'algoritmo a un'architettura GPU
- 3) Utilizziamo istruzioni veloci in HW

PROBLEMA: METODO DI MARSIGLIA

Warp Divergence and Unused Values

```
if (st.hasSpare) {  
    st.hasSpare = false;  
    return st.spare;  
}
```

generated points
thrown away = $1 - \pi / 4 = 21.46\%$

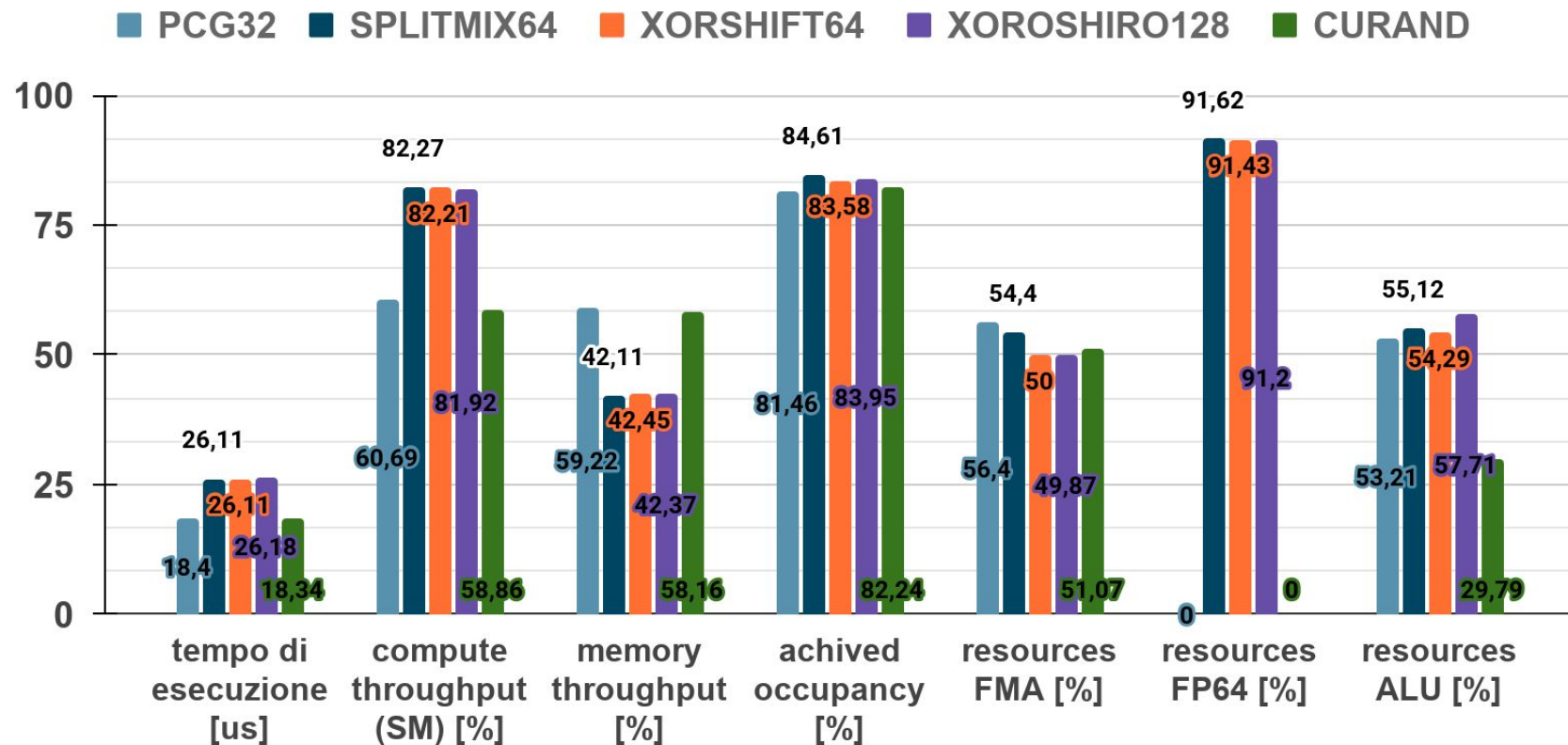
```
float x, y, s;  
do {  
    x = 2.0f * myxs64_u01(myxs64_step(st.s)) - 1.0f;  
    y = 2.0f * myxs64_u01(myxs64_step(st.s)) - 1.0f;  
    s = x*x + y*y;  
} while (s >= 1.0f || s == 0.0f);  
float m = sqrtf(-2.0f * logf(s) / s);  
st.spare = y * m;  
st.hasSpare = true;  
return x * m;
```

SOLUZIONE: METODO BOX-MULLER

It transforms two uniformly distributed variables directly into two normally distributed variables using a fixed sequence of math operations (logarithm, square root, sine, and cosine)

```
float u1 = myxs64_u01(myxs64_step(st.s));  
float u2 = myxs64_u01(myxs64_step(st.s));  
// assicuriamoci che u1≠0 per evitare log(0) = -inf  
u1 = fmaxf(u1, 5.9604644775390625e-08f);  
  
float r = __fsqrt_rn(-2.0f * __logf(u1));  
float s, c;  
  
__sincosf(6.28318530717958647692f * u2, &s, &c);  
  
return make_float2(r * c, r * s);
```

CONFRONTIAMO LE DIVERSE PERFORMANCE



COME VALUTIAMO LA QUALITÀ STATISTICA DELLE SEQUENZE RANDOM?

- 1) Generiamo sequenze di 512 MB di uint64
- 2) Verifichiamo ogni sequenza con appositi tool
- 3) Confrontiamo i risultati dei test tra di loro

TESTER USATI

- **Dieharder** (George Marsiglia, 1995)

Esegue **batterie di test statistici** sulla qualità degli RNG

- **PractRand**

Nuovo standard assieme a TestU01, **migliori prestazioni e test più approfonditi rispetto al passato**

- **TestU01?**

Avrebbe necessitato di refactoring dei programmi di benchmark, difficilmente implementabile con sistemi paralleli

RISULTATI DIEHARDER BENCHMARK

<i>RNG</i>	PASSED	WEAK	FAILED
<i>CuRAND</i>	110	4	0
<i>PCG32</i>	51	17	46
<i>PCG32 (SplitMix64)</i>	107	6	1
<i>SplitMix64</i>	108	5	1
<i>Xoroshiro128+</i>	48	20	46
<i>Xoroshiro128+ (SplitMix64)</i>	107	5	2
<i>Xorshift64*</i>	79	16	19
<i>Xorshift64* (SplitMix64)</i>	111	3	0

RISULTATI PRACTRAND BENCHMARK

<i>RNG</i>	NO ANOMALIES	SUSPICIOUS	VERY SUSPICIOUS	FAIL	SERIOUS FAIL
<i>CuRAND</i>	205	0	0	0	0
<i>PCG32</i>	108	5	7	12	73
<i>PCG32 (SplitMix64)</i>	205	0	0	0	0
<i>SplitMix64</i>	205	0	0	0	0
<i>Xoroshiro128+</i>	19	1	2	15	155
<i>Xoroshiro128+ (SplitMix64)</i>	157	7	5	13	23
<i>Xorshift64*</i>	52	8	12	6	114
<i>Xorshift64* (SplitMix64)</i>	205	0	0	0	0

GRAZIE MILLE PER L'ATTENZIONE



Maurizio Amadori, matricola 0001243119
Andrea Simonetti, matricola 0001221864

Sistemi di Elaborazione Accelerata M, 30/01/2026

[GitHub](#)